



DPUMultiSwitch

Dies This DPU provides the possibility to switch one of its In signals to the output based on a Switch input which can be defined independently for each In. Each In channel can be parametrized with independent Multiplier, Divisor and an Offset.

1. Inputs:

<i>In</i> [1...10]	Input values which are applied to the Multiplier, Divisor and Offset as specified below. Default: 0
<i>Switch</i> [1...10]	Trigger values used to select the In value with the same index. Default = 0
<i>Multiplier</i> [1...10]	Multiplier applied to InX if it is selected by the corresponding SwitchX. Default = 1
<i>Divisor</i> [1...10]	Divisor applied to InX if it is selected by the corresponding SwitchX. Default = 1
<i>Offset</i> [1...10]	Offset applied to InX if it is selected by the corresponding SwitchX. Default = 0
<i>OutMin</i>	Minimum output value after calculations. Default: -1e300
<i>OutMax</i>	Maximum output value after calculations. Default: 1e300
<i>OutDefault</i>	Default output value if no switch is 1. Default: 0

2. Outputs:

<i>Out</i>	Output of the DPU
------------	-------------------

3. Functional principle:

The DPU checks the Switch variables from 0 to 10. If SwitchX equals 1, the equation below is used to calculate the output:

$$Out = \frac{In \cdot Multiplier}{Divisor} + Offset$$

The output value is limited to the range [OutMin, OutMax]. If no Switch input is 1, the output is set to OutDefault.



4. Example:

This DPU works well using the following script:

```
Pool MultiSwitchKeyboard
{
    Executable = true;
    DPUKeyboardState keyboard
    {
        Computer = {LOCALHOST};
        Index = 1;
    };

    DPUMultiSwitch switch
    {
        Computer = {LOCALHOST};
        Index = 2;

        In1 = 1;
        In2 = 2;
        In3 = 3;
    };

    Connections =
    {
        keyboard.A -> switch.Switch1,
        keyboard.S -> switch.Switch2,
        keyboard.D -> switch.Switch3
    };
};
```